

UTXO-VM Technical Specification

Junkcoin Cross-Chain Settlement Layer
Protocol & Architecture Document

UTXO-VM Development Team

September 3, 2026

Abstract

This document specifies the UTXO-VM protocol, a chain-agnostic smart object execution framework for UTXO-based blockchains. UTXO-VM enables Junkcoin (JKC) to function as a cross-chain settlement layer for AuxPow chains through a dual execution model: native opcodes for performance-critical operations and inscription envelopes for universal compatibility. The protocol achieves Turing-complete execution while maintaining the security properties of UTXO-based accounting through Single-Use Seals.

Contents

1	Executive Summary	2
1.1	Key Metrics	2
2	Architecture Overview	2
2.1	System Components	2
2.2	Data Flow	2
3	Protocol Specification	3
3.1	Wire Format: Inscription Envelope	3
3.2	ScriptPubKey Encoding	3
3.3	Content Types	3
4	Opcode Specification	3
4.1	JKC Native Opcodes	3
4.2	Opcode Format	3
5	State Management	4
5.1	Single-Use Seals	4
5.2	State Transition Record	4
6	Cross-Chain Settlement	4
6.1	Settlement Architecture	4
6.2	Settlement Process	5
7	Core Components	5
7.1	Core-VM (Rust)	5
7.2	Indexer (TypeScript)	5
7.3	SQLite Database	5
8	API Reference	6
8.1	Endpoints	6

9	Deployment	6
9.1	Docker	6
9.2	Environment Variables	6
10	Security Considerations	6
11	Future Work	7
A	Test Results	7
B	Glossary	7

1 Executive Summary

UTXO-VM is a clean-room, patent-free smart object execution framework that enables:

- **Cross-chain settlement:** Junkcoin serves as the settlement layer for AuxPow UTXO chains
- **Dual execution model:** Native opcodes (primary) + inscription envelopes (fallback)
- **Universal compatibility:** Works on any Bitcoin-like chain without protocol changes
- **State management:** Single-Use Seals for deterministic state transitions
- **WASM execution:** Wasmtime-based runtime for smart contracts

1.1 Key Metrics

Metric	Value
Testnet Transactions	28+
Smart Objects Indexed	25+
Test Pass Rate	100% (13/13)
Supported Chains	13
Opcodes Implemented	5
Block Height	164,900+

Table 1: Current Implementation Status

2 Architecture Overview

2.1 System Components

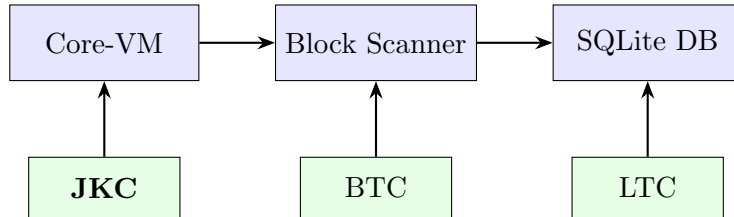


Figure 1: UTXO-VM System Architecture

2.2 Data Flow

1. **Block Ingestion:** Scanner fetches blocks from electrs API
2. **Transaction Parsing:** OP_RETURN outputs analyzed for envelopes/opcodes
3. **Execution:** Envelopes processed via inscription parser; opcodes via native handler
4. **State Update:** Single-Use Seals spent and created for state transitions
5. **Persistence:** State stored in SQLite with WAL mode

3 Protocol Specification

3.1 Wire Format: Inscription Envelope

The UTXO-VM protocol uses Bitcoin's OP_RETURN and witness script mechanisms to embed data:

```
1 OP_FALSE OP_IF
2   OP_PUSH "utxovm"           # Protocol Identifier (6 bytes)
3   OP_PUSH <version>          # Protocol Version (1 byte)
4   OP_PUSH <contentType>      # MIME Type (variable)
5   OP_PUSH <payload>          # Contract/Call Data (variable)
6 OP_ENDIF
```

Listing 1: Envelope Structure

3.2 ScriptPubKey Encoding

```
1 # OP_RETURN + PushData + Envelope
2 6a                               # OP_RETURN
3 4c 2d                           # PUSHDATA1, 45 bytes
4 00                               # OP_FALSE
5 63                               # OP_IF
6 06 7574786f766d                 # PUSH "utxovm" (6 bytes)
7 01 01                           # PUSH version 1
8 10 6170706c696361746966...      # PUSH "application/json" (16 bytes)
9 0f 7b226e616d65223a22...        # PUSH payload (15 bytes)
10 68                             # OP_ENDIF
```

Listing 2: Example: Deploy Contract

3.3 Content Types

Content Type	Purpose
application/wasm	Deploy WASM smart contract
application/json	Execute method call
application/octet-stream	Raw data inscription

Table 2: Supported Content Types

4 Opcode Specification

4.1 JKC Native Opcodes

When Junkcoin enables opcode support, the following opcodes are available:

4.2 Opcode Format

```
1 # ScriptPubKey for OP_CREATE_SMART_OBJECT
2 c0                               # Opcode: CREATE_SMART_OBJECT
3 {                               # JSON Payload
4   "codeHash": "0x...",
5   "wasmHex": "0061736d...",
6   "owner": "address",
7   "satoshis": "1000",
8   "initialState": {...}
```

Opcode	Name	Input	Output
0xc0	OP_CREATE_SMART_OBJECT	JSON	Object ID
0xc1	OP_CALL_SMART_OBJECT	JSON	State Hash
0xc2	OP_UPDATE_STATE	JSON	State Hash
0xc3	OP_SETTLE_CROSS_CHAIN	JSON	Settlement ID
0xc4	OP_VERIFY_SEAL	JSON	Boolean

Table 3: JKC Opcode Table

9 }

Listing 3: Opcode Structure

5 State Management

5.1 Single-Use Seals

Every smart object is bound to a UTXO via a **Single-Use Seal**:

- **Seal Format:** txid:vout (e.g., abc123...:0)
- **State Transition:** Spending a seal creates a new seal
- **Deterministic:** Same input state + method = same output state

5.2 State Transition Record

```

1 {
2   "objectId": "obj_abc123...",
3   "txid": "def456...",
4   "consumedSeal": "abc123...:0",
5   "newSeal": "def456...:0",
6   "method": "transfer",
7   "events": [
8     {
9       "topic": "Transfer",
10      "data": "1000 tokens transferred"
11    }
12  ],
13  "blockHeight": 164900,
14  "timestamp": 1693776000000
15 }
```

Listing 4: State Transition

6 Cross-Chain Settlement

6.1 Settlement Architecture



Figure 2: Cross-Chain Settlement Flow

6.2 Settlement Process

1. **Child Chain:** Creates state anchor with Merkle proof
2. **Anchor Transaction:** Broadcast to JKC via OP_SETTLE_CROSS_CHAIN
3. **Verification:** JKC validates Merkle proof and state root
4. **Settlement:** State recorded on JKC settlement layer
5. **Claim:** Child chain can claim settled state

7 Core Components

7.1 Core-VM (Rust)

The WASM execution engine built with Wasmtime:

```
1 pub fn execute(  
2     wasm_bytes: &[u8],  
3     state: &[u8],  
4     caller: &str,  
5     method: &str,  
6     args: &[u8],  
7 ) -> Result<ExecutionResult, VMError> {  
8     // 1. Validate WASM module  
9     // 2. Instantiate with host functions  
10    // 3. Execute method with gas metering  
11    // 4. Return updated state  
12 }
```

Listing 5: VM Execution

7.2 Indexer (TypeScript)

Block scanner with electrs integration:

```
1 async processTransaction(tx: BlockTransaction) {  
2     // Check for opcodes first (JKC)  
3     const opcode = detectOpcode(tx.scriptPubKey);  
4     if (opcode) {  
5         return processOpcode(opcode, tx);  
6     }  
7  
8     // Fallback to inscription parsing  
9     const envelope = parseEnvelope(tx.opReturn);  
10    if (envelope) {  
11        return processEnvelope(envelope, tx);  
12    }  
13 }
```

Listing 6: Scanner Logic

7.3 SQLite Database

Persistent storage with WAL mode:

Table	Purpose
objects	Smart object records
state_transitions	State transition history
sync_state	Chain sync status

Table 4: Database Schema

8 API Reference

8.1 Endpoints

Method	Endpoint	Description
GET	/api/v1/chain/info	Chain information
GET	/api/v1/chains	Available chains
GET	/api/v1/objects	All smart objects
GET	/api/v1/object/:id	Get object by ID
GET	/api/v1/object/:id/history	State transitions
POST	/api/v1/sync	Sync from electrs
GET	/api/v1/sync/status	Sync status
POST	/api/v1/mempool/scan	Scan mempool
POST	/api/v1/parse-envelope	Parse envelope
POST	/api/v1/broadcast	Broadcast transaction

9 Deployment

9.1 Docker

```

1 # Testnet
2 CHAIN=JKC_TESTNET docker-compose up -d
3
4 # Mainnet (when opcodes enabled)
5 docker-compose --profile mainnet up -d

```

Listing 7: Docker Compose

9.2 Environment Variables

Variable	Default
CHAIN	JKC_TESTNET
CHAIN_CONFIG_PATH	./chain.json
DB_PATH	./data/indexer.db
SYNC_INTERVAL_MS	30000
PORT	9773

Table 6: Environment Configuration

10 Security Considerations

- **Deterministic Execution:** Same input always produces same output
- **Gas Metering:** Prevents infinite loops and DoS attacks
- **Single-Use Seals:** Prevents double-spending of state
- **Merkle Proofs:** Verifiable cross-chain state anchoring
- **WASM Sandboxing:** Isolated execution environment

11 Future Work

1. **Mainnet Opcodes:** Enable native opcodes on JKC mainnet
2. **Child Chain Integration:** Connect BTC, LTC, DOGE settlement
3. **Privacy Extensions:** MWEB integration for confidential transactions
4. **Mempool Indexing:** Real-time unconfirmed transaction tracking
5. **Production Monitoring:** Observability and alerting

A Test Results

Test Suite	Result
Token E2E	6/6 PASS
Sequential Chain	11/11 PASS
Comprehensive Suite	13/13 PASS
Cross-Chain Anchor	7/7 PASS
Persistence	12/12 PASS
Total	49/49 PASS

Table 7: Test Results Summary

B Glossary

AuxPow Auxiliary Proof of Work - merge mining mechanism

Single-Use Seal Cryptographic primitive that can only be consumed once

Envelope Data structure embedded in OP_RETURN outputs

Opcode Native instruction executed by the blockchain

Settlement Process of anchoring state from child chain to settlement layer

WASM WebAssembly - binary instruction format for stack-based virtual machines